Chapter - 1
Learning From Data

Aviral Janveja

1 Introduction

Machine Learning is about learning patterns from data, in scenarios where explicit programming is not feasible. For example, we learn to recognize trees not through a written description, but by repeatedly looking at them, either in real life or through pictures. Over time, we learn to identify their distinct physical features. Similarly, machine learning models work by learning patterns directly from example data-points.

2 The Machine Learning Process

Let us look at the main elements of the machine learning process, using an example from the finance domain. Suppose a customer applies for a loan. The bank must now decide whether granting the loan is a good idea or not, since their goal is to maximize profit while minimizing risk. Naturally, the bank has no magic formula to predict whether a customer will make timely repayments or not. However, it is still possible to check similar customer data from the past, and see how their repayment behavior turned out to be.

The idea is to try and uncover the underlying pattern that connects customer information with their repayment behavior, using past data-points. Hopefully, we can then use it to predict the creditworthiness of fresh applicants. Using this example, let us now outline the main elements of the machine learning process :

2.1 The Underlying Pattern

In machine learning, our goal is to uncover the underlying pattern from the given set of data-points. In the loan example from above, this underlying pattern is the relationship that connects customer information with their creditworthiness. Although this relation is unknown to us, we do see glimpses of it through the available data-points.

So, how should we describe this unknown pattern mathematically? Is it a deterministic function that always produces the same output for a given input, or is it inherently probabilistic? To answer this, let us consider a simple question :

> **Can two loan applicants with identical information exhibit different repayment behaviour?**

Well, it is entirely possible and in fact quite common in real-world scenarios. Two applicants may have identical ages, salaries and employment histories, yet one eventually repays the loan while the other defaults. This can happen because of factors that are not recorded in our data, such as future job loss, unexpected medical expenses, or simply the inherent uncertainty present in human behaviour. This leads us to an important observation :

> **Most real-world scenarios are practically probabilistic rather than deterministic in nature. Hence, we will build our framework accordingly to capture that, from the beginning.**

Therefore, instead of assuming a single fixed output for every input, we describe the underlying pattern in terms of a list or a map, which tells us how likely each possible outcome is for a given input. Formally, this is know as the conditional probability distribution, commonly called the **target distribution** :

$$P(y \mid \mathbf{x})$$

It represents the probability of observing the output y given the input $\mathbf{x}$. For example,

Target Distribution Vs Target Function

Let us say the bank has a strict rule : anyone with income above 7 lakhs per year gets approved, else is denied. This is a deterministic rule, there is no randomness here. Hence, we can represent it formally using a function, called a **target function** in machine learning. For a given input salary, let us say $x = 7.2$ in lakhs per year. The target function will output :

$$y = f(7.2) = +1 \text{ (loan approved)}$$

However, in real life, two people can both have a salary of 7 lakhs per year, but one might lose their job next week while the other gets a promotion. The salary data-point alone cannot predict the future, it only gives a trend. Hence, for an input $x = 7.2$, we get a **target distribution** instead :

$$P(y = +1 \mid x = 7.2) = 0.80 \text{ (80\% chance of repayment)}$$
$$P(y = -1 \mid x = 7.2) = 0.20 \text{ (20\% chance of default)}$$

This distribution captures real world factors like unmeasured variables, human randomness or measurement errors, also called **noise**. Finally, note that the target function is not a different concept, it is simply a special case of the target distribution with zero noise.

2.2 The Data Set

We do not see the target function / distribution as it is unknown to us. That is why machine learning, we only get a glimpse of that throught the available data points.

As discussed above, we are trying to learn patterns from data. Therefore, without sufficient data-points, the machine learning process cannot even begin. In line with the above example, the data from N past customers is described as follows :

$$\textbf{Data} : (\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), (\mathbf{x}_3, y_3)...(\mathbf{x}_N, y_N)$$

Here $\mathbf{x}_n$ is a column matrix representing customer information such as age, salary, years in residence, current debt and so on. Whereas $y_n \in \{+1, -1\}$ is a binary value representing whether the customer was creditworthy or not in hindsight.

2.3 The Learning Model

Combining the learning model with the Error Measures in terms of the target distribution view.

The learning model defines the **hypothesis set** and the **learning algorithm**. The hypothesis set H is a set of hypothesis functions $\{h_1, h_2, h_3...h_m\}$ out of which the learning algorithm picks the **final hypothesis function** h_f based on the available data-points.

This final hypothesis function is our approximation of the unknown target function. The goal in machine learning is that h_f approximates f well, that is :

$$h_f \approx f$$

An an example, we examine a simple learning model called Perceptron next, to understand how the above process works, continuing with the bank-loan example.

3 Perceptron

The learning algorithm tries to output a hypothesis $h(x)$ that attempts to predict the most likely y from that distribution, minimizing the expected risk.

Linear classifiers such as the perceptron do not model the entire underlying target distribution. Instead, they optimize a linear decision boundary as per the labels assigned to the past data-points. It can be thought of as the perceptron approximating the target distribution, by learning the most probable label for a given x.

For example, let us say the underlying probability distribution given a point x is as follows :

$$P(y = +1 \mid \mathbf{x}) = 0.8$$

and

$$P(y = -1 \mid \mathbf{x}) = 0.2$$

Then, final label $y = +1$ and this what the perceptron will try to learn and optimize for, without trying to model the exact underlying probability distribution given above. This is because perception is a simple linear model. Further there will be other advanced models that will try to capture the probabilistic aspect explicitly.

Let us begin by looking at the given data, which is available to us in the form of N input-output pairs :

$$(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), (\mathbf{x}_3, y_3)...(\mathbf{x}_N, y_N)$$

As discussed, $\mathbf{x}_n$ is the application information of the n^{th} customer, which is represented by a column matrix as shown below :

$$\mathbf{x}_n = \begin{bmatrix} x_1 \\ x_2 \\ . \\ . \\ x_d \end{bmatrix}$$

Here x_1, x_2 and so on, upto x_d are the attributes of a customer like age, salary, years in residence, outstanding debt and so on. Whereas $y_n \in \{+1, -1\}$ is binary value representing the creditworthiness of a past customer in hindsight.

3.1 Perceptron Hypothesis Function

Now, what a perceptron basically does, is to assign weights to each of the customer attributes. These weights are multiplied to their respective attributes and then summed-up as follows :

$$w_1x_1 + w_2x_2 + ... + w_dx_d$$

The above linear summation of weights and attributes can be called the credit score of an individual customer. The idea is pretty simple, If the credit score is greater than a certain threshold, then we approve the loan, else we deny it :

$$w_1x_1 + w_2x_2 + ... + w_dx_d > \text{threshold (Approve Loan)}$$
$$w_1x_1 + w_2x_2 + ... + w_dx_d < \text{threshold (Deny Loan)}$$

This is equivalent to :

$$w_1x_1 + w_2x_2 + ... + w_dx_d - \text{threshold} > 0 \text{ (Approve Loan)}$$
$$w_1x_1 + w_2x_2 + ... + w_dx_d - \text{threshold} < 0 \text{ (Deny Loan)}$$

We can simplify the above expression further, by taking the minus of threshold as the zero$^{\text{th}}$ weight ($w_0 = -\text{threshold}$) and introducing an artificial attribute $x_0 = 1$ to go along with it, hence giving us :

$$w_0x_0 + w_1x_1 + w_2x_2 + ... + w_dx_d > 0 \text{ (Approve Loan)}$$
$$w_0x_0 + w_1x_1 + w_2x_2 + ... + w_dx_d < 0 \text{ (Deny Loan)}$$

Next, we re-write the above expression in vector (column matrix) notation to make it shorter and cleaner :

$$w_0x_0 + w_1x_1 + .. + w_dx_d = \begin{bmatrix} w_0 & w_1 & . & . & w_d \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ . \\ . \\ x_d \end{bmatrix} = \mathbf{w^T x}$$

The above formula outputs a real value that could be greater than, equal to or less than zero. In order to output a binary value, similar to y, we wrap the above formula inside a **sign** function.

$$\text{sign}(\mathbf{w}^\mathbf{T}\mathbf{x})$$

The sign is just a simple function that takes in a real number value and outputs $+1$ if the value is greater than zero and -1 if it is less than or equal to zero, hence giving us the final form of our perceptron hypothesis function.

3.2 Perceptron Learning Algorithm

Examining the perceptron hypothesis function, that we have arrived at above :

$$h(\mathbf{x}) = \text{sign}(\mathbf{w}^\mathbf{T}\mathbf{x})$$

We see that $\mathbf{x}$ representing the customer information is already a given, along with y, which is the correct loan decision in hindsight. So, the underlying pattern that we are trying to learn will be reflected in our choice of weights, as represented by the weight vector $\mathbf{w}$. Hence, our task now is to arrive at a set of weights, that are able to classify all given data-points correctly.

So, how do we compute the correct weights? First, we initialize the weights randomly, let us say initialize them all to zero. The perceptron learning algorithm then iterates through the dataset, looking for misclassified points $(\mathbf{x}_n, y_n)$, where :

$$h(\mathbf{x}) \neq f(\mathbf{x})$$

that is;

$$\text{sign}(\mathbf{w}^\mathbf{T}\mathbf{x}_n) \neq y_n$$

Whenever such a misclassified point is found, the weight vector is updated by applying the following **update rule** :

$$\mathbf{w}_{new} = \mathbf{w} + y_n\mathbf{x}_n$$

This process continues, until no misclassified points remain in the dataset. And that is it, we then have our required set of weight values.

3.3 Update Rule Justification

Let us examine how the above defined update rule actually nudges the perceptron classifier in the right direction at each misclassified point. If a point $(\mathbf{x}_n, y_n)$ is misclassified, it means :

$$\text{sign}(\mathbf{w}^\mathbf{T}\mathbf{x}_n) \neq y_n$$

Accordingly, the perceptron updates the weight vector as follows :

$$\mathbf{w}_{new} = \mathbf{w} + y_n\mathbf{x}_n$$

Substituting the updated weight vector from above and ignoring the sign function for now, we get the following :

$$\mathbf{w}_{new}^\mathbf{T}\mathbf{x}_n = (\mathbf{w} + y_n\mathbf{x}_n)^\mathbf{T}\mathbf{x}_n$$

Expanding the right-hand side, we get :

$$\mathbf{w}_{new}^{\mathbf{T}}\mathbf{x}_n = \mathbf{w}^{\mathbf{T}}\mathbf{x}_n + y_n\mathbf{x}_n^{\mathbf{T}}\mathbf{x}_n$$

This is equivalent to :

$$\mathbf{w}_{new}^{\mathbf{T}}\mathbf{x}_n = \mathbf{w}^{\mathbf{T}}\mathbf{x}_n + y_n||\mathbf{x}_n||^2$$

Therefore, for a misclassified point, where $y_n = +1$ and $\text{sign}(\mathbf{w}^{\mathbf{T}}\mathbf{x}_n) = -1$, it implies $\mathbf{w}^{\mathbf{T}}\mathbf{x}_n \leq 0$ and after the update we get :

$$\mathbf{w}_{new}^{\mathbf{T}}\mathbf{x}_n = \mathbf{w}^{\mathbf{T}}\mathbf{x}_n + ||\mathbf{x}_n||^2$$

The update is adding some positive value to $\mathbf{w}^{\mathbf{T}}\mathbf{x}_n$ which might make $\mathbf{w}^{\mathbf{T}}\mathbf{x}_n > 0$. Although, it does not guarantee an immediate sign flip, it clearly nudges the perceptron classifier in the right direction.

Similarly, when $y_n = -1$ and $\text{sign}(\mathbf{w}^{\mathbf{T}}\mathbf{x}_n) = +1$, implying $\mathbf{w}^{\mathbf{T}}\mathbf{x}_n > 0$, we get :

$$\mathbf{w}_{new}^{\mathbf{T}}\mathbf{x}_n = \mathbf{w}^{\mathbf{T}}\mathbf{x}_n - ||\mathbf{x}_n||^2$$

Hence, showing that each update to the weight vector nudges the perceptron in the required direction, improving its performance on that particular data-point. The following image illustrates the above discussion :

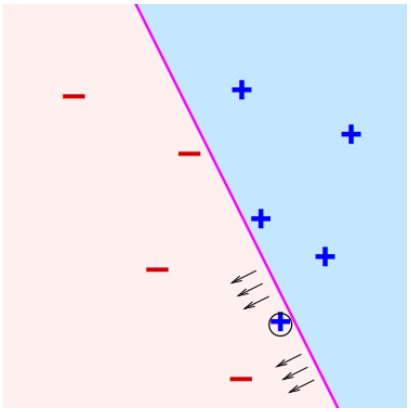

Figure 1: Perceptron

The pink line represents our linear perceptron classifier that is uniquely determined by the choice of weight vector $\mathbf{w}$. We have a misclassified **plus** point in the **minus** region and the update-rule tries to correct this by nudging the classifier in the required direction.

3.4 Convergence Theorem

The convergence theorem for perceptron, guarantees that if the data is **linearly separable**, repeated updates will eventually classify all points correctly.

Data being linearly separable simply means that, when the data-points are plotted on a two-dimensional graph, it is possible to draw a straight-line that separates them into two distinct categories. The data-points in the above image for example, are linearly separable.

4 Types of Learning

As discussed in the introduction, machine learning is about learning patterns from data. Alternatively, we can put it as **using a set of observations to uncover an underlying process**. In either case, we have established that having a set of observations or data-points is a non-negotiable. However, the form in which the data is available to us can vary. Depending on this form and the nature of the learning task, machine learning is commonly divided into three main paradigms: supervised learning, unsupervised learning, and reinforcement learning.

4.1 Supervised Learning

In supervised learning, the model is trained on labeled data, where each example consists of : (**input, correct output**). The goal is to approximate a function from inputs to outputs that generalizes well to unseen data. For instance, the perceptron model above.

4.2 Unsupervised Learning

In unsupervised learning, the data is in the form of inputs only, without associated labels : (**input, no output**). The task is to discover hidden structures, patterns or clusters within the data. Examples include clustering, dimensionality reduction, and generative modeling.

4.3 Reinforcement Learning

In reinforcement learning, the model interacts with the environment by taking actions and receiving feedback in the form of rewards or penalties : (**action, feedback**). The model learns by trial and error to maximize its cumulative reward, gradually improving its decision-making. This setup is especially suited to sequential problems such as game playing, robotics and control systems.

5 References

1. CalTech Machine Learning Course - CS156, Lecture 1.
2. CalTech Machine Learning Course - CS156, Lecture 4.